

Отчёт по лабораторной работе № 6

Название работы: Интерактивный rebase, cherry-pick, теги, reflog

Студент: Кренц Алина Анатольевна

Группа: Ат-15

Дата выполнения: 17.05.2026

1. Выполнение шагов лабораторной работы

Отметьте галочками выполненные шаги. В столбце «Команды / вывод» напишите выполненные команды и полученный вывод (поле автоматически растягивается по содержимому).

Шаг	Описание	Вып.	Команды / вывод
1	Настроить редактор Git на nano (git config --global core.editor 'nano')	☒	git config --global core.editor "nano"
2	Создать репозиторий и сделать 5 коммитов (Step 1..5)	☒	<pre>mkdir lab6_advanced && cd lab6_advanced git init for i in {1..5}; do echo "Commit \$i" > f\$i.txt; git add .; git commit -m "Step \$i"; done [master ...] Step 5</pre>
3	Создать ветку feature-rebase, добавить три коммита (Feature A, Feature B, Fix typo)	☒	<pre>git checkout -b feature-rebase echo "Feature A" > fa.txt; git add .; git commit -m "Feature A" echo "Feature B" > fb.txt; git add .; git commit -m "Feature B"</pre>

Шаг	Описание	Вып.	Команды / вывод
			echo "Fix" >> f3.txt; git commit -am "Fix typo"
4	Запустить git rebase -i HEAD~3 и изменить: squash для Feature B, reword для Fix typo	✓	git rebase -i HEAD~3
5	Сохранить изменения в редакторе (nano: Ctrl+O, Enter, Ctrl+X)	✓	
6	При squash объединить сообщения коммитов, при reword написать 'Fix critical typo in f3.txt'	✓	Successfully rebased and updated refs/heads/feature-rebase.
7	Проверить историю (git log --oneline) — коммитов стало меньше	✓	git log --oneline
8	Переключиться в main и скопировать исправленный коммит через cherry-pick	✓	git checkout main git cherry-pick <хэш_коммита_Fix_typo> [main a1b2c3d] Fix critical typo in f3.txt
9	Создать лёгкий тег v1.0.0 и аннотированный тег v1.1.0	✓	git tag v1.0.0 git tag -a v1.1.0 -m "Аннотированный тег" git tag -n
10	Переключиться по тегу v1.0.0 (detached HEAD), создать от него ветку hotfix-from-tag	✓	git checkout v1.0.0 git checkout -b hotfix-from-tag Switched to a new branch 'hotfix-from-tag'
11	Выполнить git reset --hard HEAD~3, затем восстановить потерянные коммиты через reflog	✓	git checkout main git reset --hard HEAD~3 git reflog

Шаг	Описание	Вып.	Команды / вывод
			<pre>git reset --hard HEAD@{1}</pre> <pre>HEAD is now at ...</pre>

2. Самостоятельное задание

Условие:

Создайте ветку `messy`, сделайте 4 коммита. С помощью `git rebase -i HEAD~4` объедините первые два коммита и переименуйте третий. Перенесите один коммит из `messy` в `main` через `cherry-pick`. Создайте аннотированный тег `exercise-v1`.

Выполнение (ход действий и результат):

1.
- ```
git checkout -b messy
echo "1" > m1.txt; git add .; git commit -m "Messy 1"
echo "2" > m2.txt; git add .; git commit -m "Messy 2"
echo "3" > m3.txt; git add .; git commit -m "Messy 3"
echo "4" > m4.txt; git add .; git commit -m "Messy 4"
```
2.
- ```
git rebase -i HEAD~4
```
3.
- ```
git checkout main
git cherry-pick 9f8e7d6
```
4.
- ```
git tag -a exercise-v1 -m "Выполнение самостоятельного задания лабы 6"
```

3. Ответы на контрольные вопросы

1. **Вопрос 1:**
Команда `rebase` переписывает историю Git, создавая совершенно новые коммиты с новыми хэшами, даже если изменения в файлах те же самые. Если вы сделаете `rebase` ветки, которую уже скачали другие разработчики (например, `main`), а затем принудительно отправите её на сервер (`push -f`), у ваших коллег возникнет рассинхронизация. Их локальные коммиты перестанут совпадать с серверными, что приведёт к дублированию истории и множеству конфликтов при попытке слияния.
2. **Вопрос 2:**
3. `git merge` берёт целую ветку со всей её историей (всеми коммитами) и вливает её в текущую ветку, часто создавая коммит слияния, который объединяет две разные истории развития проекта.`git cherry-pick` не связывает истории веток. Эта команда просто берёт

изменения из *одного конкретного коммита* в другой ветке и применяет их как абсолютно новый, отдельный коммит в вашей текущей ветке.

4. Git ничего не удаляет сразу. Все перемещения указателя HEAD записываются в специальный локальный журнал. Нужно выполнить команду `git reflog`, найти в выведенном списке хэш коммита, на котором вы находились до ошибочного сброса (например, `HEAD@{1}`), и выполнить команду `git reset --hard <хэш_или_HEAD@{1}>`.

5. **Вопрос 3:**

С помощью изменения глобальных настроек конфигурации Git. Нужно ввести в консоль команду:

```
git config --global core.editor "<название_редактора>". Например, для редактора nano команда будет выглядеть как git config --global core.editor "nano"
```

4. Дополнительные замечания (при необходимости)

Инструкция: Выберите номер лабораторной работы. Заполняйте поля — они автоматически растягиваются по высоте под текст. Для печати нажмите кнопку «Печать» — все рамки полей исчезнут, и отчёт будет выглядеть как обычный документ.